Algorithms & Computability — Practice Tutorial 1

Same exercise types as Tutorial 1, different content.
Study the solutions using the 3-pass method.

How to use this sheet:

1. **Pass 1:** Read each exercise, then read its solution line by line. For every line, ask yourself: *why is this line here?*
2. **Pass 2:** Cover the solution. Try to reproduce it on blank paper. When you get stuck, peek, then close and continue.
3. **Pass 3:** Go back to the *original* Tutorial 1 exercises (without solutions). Try those. They should feel doable now.

Exercise 1: Test Your Understanding

Answer the following questions and justify your answers.

1. Can the input alphabet Σ contain the blank symbol $\sqcup$?
2. Can a DTM have exactly two states?
3. Can the transition function δ ever map to the reject state r?
4. If a DTM reaches the accept state t, can it continue computing?
5. Suppose a DTM is in state $q \neq t, r$ and reads symbol a. Can the machine transition to the same state q while writing the same symbol a?
6. Is it possible for a DTM to never halt on any input?

Solution to Exercise 1

The method: For each question, find the relevant part of the DTM definition $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with $\delta : (Q \setminus \{t, r\}) \times \Gamma \to Q \times \Gamma \times \{-1, +1\}$ and answer by citing it.

1. **No.** By definition, $\sqcup \in \Gamma$ but $\sqcup \notin \Sigma$. The input alphabet and tape alphabet are related by $\Sigma \subset \Gamma$ (strict subset) precisely because Γ contains $\sqcup$ and Σ does not.

   Why this answer works: I looked at the definition, found where Σ and $\sqcup$ are mentioned, and cited the constraint.

2. **Yes.** The definition requires $t \in Q$ and $r \in Q$ with $t \neq r$. So the minimum is 2 states: $Q = \{t, r\}$. But note: in this case, s (the start state) must be either t or r. If $s = t$, the machine accepts every input immediately. If $s = r$, it rejects every input immediately. Both are valid (but boring) DTMs.

   Why this answer works: I checked the constraints on Q ($t \neq r$, both in Q, $s \in Q$) and found the minimum.

3. **Yes.** The codomain of δ is $Q \times \Gamma \times \{-1, +1\}$, and $r \in Q$. So $\delta(q, a) = (r, b, d)$ is perfectly valid. This is how a DTM rejects — it transitions into state r.

   Why this answer works: I looked at the type signature of δ and checked if r is in the codomain.

4. **No.** The transition function δ is defined on $(Q \setminus \{t, r\}) \times \Gamma$. State t is explicitly excluded from the domain. Once the machine reaches t, there is no transition to take — the computation stops.

   Why this answer works: I looked at the domain of δ and noticed t is excluded.

5. **Yes.** Nothing in the definition prevents $\delta(q, a) = (q, a, d)$. The machine stays in the same state, writes the same symbol (leaving the tape unchanged), and moves in direction d. This is a common pattern (self-loops for scanning).

   Why this answer works: I checked if the definition imposes any restriction on the output matching the input — it doesn't.

6. **Yes.** The definition does not require a DTM to halt. A DTM halts only when it reaches state t or r. If the transitions never lead to t or r, the machine runs forever. Example: $Q = \{s, t, r\}$, $\Sigma = \{a\}$, $\Gamma = \{a, \sqcup\}$, $\delta(s, a) = (s, a, +1)$, $\delta(s, \sqcup) = (s, a, +1)$. This machine just writes a's forever and never halts.

   Why this answer works: I checked if the definition guarantees halting — it doesn't. Then I gave a concrete example.

Exercise 2: On Precise Formulations

Some of the following definitions of **computable language** are correct and some are wrong. Mark the correct definitions and for the incorrect ones, identify the part that is wrong and explain why.

1. A language L is computable if there exists a DTM M such that $L(M) = L$.
2. A language L is computable if there exists a halting DTM M such that M accepts every word $w \in L$ and rejects every word $w \notin L$.
3. A language L is computable if for every DTM M we have that M halts on all inputs and $L(M) = L$.
4. A language L is computable if there exists a halting DTM M such that M accepts every word $w \in L$.
5. A language L is computable if there exists a DTM M that halts on every input and $L(M) = L$.
6. A language L is computable if a halting DTM accepts L.

Solution to Exercise 2

The method: Write down the correct definition: "L is computable if there exists a **halting** DTM M such that $L(M) = L$." Now compare each statement word by word.

1. **WRONG:** "there exists a DTM M such that $L(M) = L$" — this is missing the word halting. Without "halting," this is just the definition of *computably-enumerable*, not computable. A CE language has a DTM that recognizes it, but the DTM might loop on inputs not in L.
2. **CORRECT.** A halting DTM that accepts words in L and rejects words not in L is exactly a halting DTM M with $L(M) = L$. This is just the definition stated in more explicit terms.
3. **WRONG:** "for every DTM M" — the definition requires "**there exists**" a DTM, not that every DTM works. This would mean every DTM in existence accepts exactly L, which is absurd.
4. **WRONG:** The statement says "M accepts every word $w \in L$" — this only guarantees $L \subseteq L(M)$, but it does NOT say $L(M) \subseteq L$. The DTM M could also accept words outside L, making $L(M) \supsetneq L$.

   In fact, this broken definition would make *every* language "computable": just take the halting DTM that accepts everything (accept all inputs immediately). It accepts every $w \in L$ for any L — but also accepts everything else. The definition is missing the requirement that M *only* accepts words in L (i.e., rejects words not in L).

   The correct version should say: "M accepts every $w \in L$ **and rejects every** $w \notin L$" or equivalently "$L(M) = L$."
5. **CORRECT.** "A DTM M that halts on every input" is the same as "a halting DTM M." Combined with $L(M) = L$, this is exactly the definition.
6. **WRONG:** "a halting DTM accepts L" — it doesn't say "there exists." Also, "a halting DTM" is vague — it could be read as "any halting DTM" or "some halting DTM." More importantly, it doesn't specify *which* DTM. The definition requires an explicit existential quantifier: "there exists a halting DTM M such that $L(M) = L$." The statement is imprecise/ambiguous rather than outright wrong, but in a formal context, this imprecision is an error.

What to learn from this exercise

The traps are always the same:

- Missing "halting" (confusing computable with CE)
- Wrong quantifier ("for every" vs "there exists")
- Missing half of the definition (only saying what happens for $w \in L$, forgetting $w \notin L$)
- Imprecise language (not naming the machine, ambiguous subjects)

Exercise 3: Turing Machines and Their Runs

Consider the following DTM $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with:

- $Q = \{s, t, r, p, q\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, \sqcup\}$
- δ given by the following table:

δ	0	1	$\sqcup$
s	$(s, 0, +1)$	$(p, 1, +1)$	$(r, \sqcup, +1)$
p	$(q, 0, +1)$	$(p, 1, +1)$	$(r, \sqcup, +1)$
q	$(q, 0, +1)$	$(q, 1, +1)$	$(t, \sqcup, +1)$

1. Give the initial configuration of M on the inputs $w_0 = 010$ and $w_1 = 111$ using the simplified notation.
2. Give the runs of M on w_0 and w_1.
3. Are w_0 and w_1 accepted or rejected by M?
4. What language does M accept? (Bonus — think about what pattern the states are tracking.)

Solution to Exercise 3

1. Initial configurations:
On $w_0 = 010$: $[s, 010\sqcup\ldots, 1]$
On $w_1 = 111$: $[s, 111\sqcup\ldots, 1]$
The initial configuration is always: state = s, tape = input followed by blanks, head at position 1.

2. Runs:
Run on $w_0 = 010$:

Step	Reasoning
$[s, 010\sqcup\ldots, 1]$	State s, read 0. $\delta(s, 0) = (s, 0, +1)$. Stay in s, write 0, move right.
$[s, 010\sqcup\ldots, 2]$	State s, read 1. $\delta(s, 1) = (p, 1, +1)$. Go to p, write 1, move right.
$[p, 010\sqcup\ldots, 3]$	State p, read 0. $\delta(p, 0) = (q, 0, +1)$. Go to q, write 0, move right.
$[q, 010\sqcup\ldots, 4]$	State q, read $\sqcup$. $\delta(q, \sqcup) = (t, \sqcup, +1)$. Go to t. **HALT — accept.**

Full run: $[s, 010\sqcup\ldots, 1] \vdash_M [s, 010\sqcup\ldots, 2] \vdash_M [p, 010\sqcup\ldots, 3] \vdash_M [q, 010\sqcup\ldots, 4] \vdash_M [t, 010\sqcup\ldots, 5]$

Run on $w_1 = 111$:

Step	Reasoning
$[s, 111\sqcup\ldots, 1]$	State s, read 1. $\delta(s, 1) = (p, 1, +1)$. Go to p.
$[p, 111\sqcup\ldots, 2]$	State p, read 1. $\delta(p, 1) = (p, 1, +1)$. Stay in p.
$[p, 111\sqcup\ldots, 3]$	State p, read 1. $\delta(p, 1) = (p, 1, +1)$. Stay in p.
$[p, 111\sqcup\ldots, 4]$	State p, read $\sqcup$. $\delta(p, \sqcup) = (r, \sqcup, +1)$. Go to r. **HALT — reject.**

Full run: $[s, 111\sqcup\ldots, 1] \vdash_M [p, 111\sqcup\ldots, 2] \vdash_M [p, 111\sqcup\ldots, 3] \vdash_M [p, 111\sqcup\ldots, 4] \vdash_M [r, 111\sqcup\ldots, 5]$

3. $w_0 = 010$ is **accepted** (run ends in t). $w_1 = 111$ is **rejected** (run ends in r).

4. What language does M accept?
Let's understand what the states track:

- s: "I've only seen 0's so far" (no 1 seen yet)
- p: "I've seen at least one 1, but no 0 after any 1"

- q: "I've seen at least one 1 followed by at least one 0"

The machine accepts (reaches $\sqcup$ in state q) when the input contains the substring 10 — i.e., a 1 followed (possibly later) by a 0. More precisely: $L(M) = \{w \in \{0,1\}^* \mid w \text{ contains the substring } 10\}$.
Check: 010 contains 10 ✓. 111 does not contain 10 (no 0 after a 1) ✓.

Exercise 4: Constructing a Simple DTM

Consider the language of words over the alphabet $\{a, b\}$ that have even length.

1. Write a formal definition of the language. Think about corner cases.
2. Give a halting DTM that accepts this language.
3. Give the run on ab.
4. Give the run on aba.
5. Give the run on the corner case(s) you considered.

Solution to Exercise 4

1. Formal definition and corner cases.
Corner cases:

- The empty word ε has length 0. Is 0 even? Yes. So ε is in the language.
- Single character words like a have length 1. Odd. Not in the language.

Formal definition: $L = \{w \in \{a, b\}^* \mid |w| \text{ is even}\} = \{w \in \{a, b\}^* \mid |w| \bmod 2 = 0\}$
Examples in L: ε, aa, ab, ba, bb, $abab$, $aabb$.
Examples not in L: a, b, aba, bbb.

2. Intuitive description of the DTM:

- Scan through the input from left to right.
- Keep track of whether we've seen an even or odd number of letters so far.
- "Even so far" and "odd so far" = two states (e and o). Start in e (zero letters seen = even).
- When we reach $\sqcup$: if in state e, accept. If in state o, reject.

Formally, $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with:

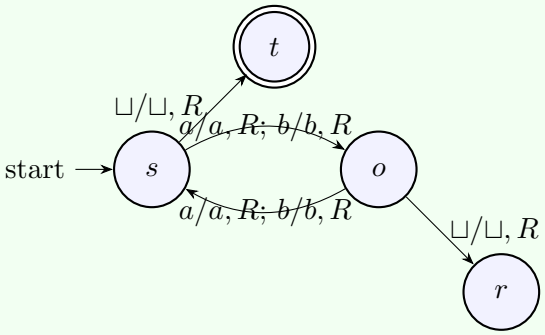
- $Q = \{s, t, r, o\}$ (we use s as the "even" state, since we start having seen 0 = even letters)
- $\Sigma = \{a, b\}$
- $\Gamma = \{a, b, \sqcup\}$
- δ given by:

δ	a	b	$\sqcup$
s	$(o, a, +1)$	$(o, b, +1)$	$(t, \sqcup, +1)$
o	$(s, a, +1)$	$(s, b, +1)$	$(r, \sqcup, +1)$

Reading the table:

- s (even count): Read any letter $\to$ count becomes odd, go to o. Read $\sqcup \to$ we've seen an even number of letters, accept.
- o (odd count): Read any letter $\to$ count becomes even, go to s. Read $\sqcup \to$ odd count, reject.

State diagram:



3. Run on ab (length 2 = even, should accept):
$[s, ab\sqcup\ldots, 1] \vdash_M [o, ab\sqcup\ldots, 2] \vdash_M [s, ab\sqcup\ldots, 3] \vdash_M [t, ab\sqcup\ldots, 4]$
State s, read a, go to o. State o, read b, go to s. State s, read $\sqcup$, accept. ✓
4. Run on aba (length 3 = odd, should reject):
$[s, aba\sqcup\ldots, 1] \vdash_M [o, aba\sqcup\ldots, 2] \vdash_M [s, aba\sqcup\ldots, 3] \vdash_M [o, aba\sqcup\ldots, 4] \vdash_M [r, aba\sqcup\ldots, 5]$
✓ Rejected.
5. Corner cases:
Run on ε: $[s, \sqcup\ldots, 1] \vdash_M [t, \sqcup\ldots, 2]$. Accepted. ✓ (length 0 = even)
Run on a: $[s, a\sqcup\ldots, 1] \vdash_M [o, a\sqcup\ldots, 2] \vdash_M [r, a\sqcup\ldots, 3]$. Rejected. ✓ (length 1 = odd)

Exercise 5: Constructing a (Slightly More Complex) DTM

Consider the language $L = \{w \in \{a, b\}^* \mid w \text{ is a palindrome}\}$.

A palindrome is a word that reads the same forwards and backwards. For example: ε, a, b, aa, bb, aba, bab, $abba$.

1. Give a halting DTM for L. Explain your solution in natural language first.
2. Give the accepting run on $aba \in L$.
3. Give the rejecting run on $ab \notin L$.

Solution to Exercise 5

1. Intuitive description:

- Read the first letter and remember it (using states q_a or q_b). Replace it with X (mark as "used").
- Scan right to the last non-X letter (= the end of the remaining input).
- Compare the last letter to the remembered first letter. If they match, mark it with X and go back to the start. If they don't match, reject.
- Repeat: the input shrinks by 2 letters each round (one from each end).
- When the input is empty (all X's) or only one letter remains, accept.

Formally, $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with:

- $Q = \{s, q_a, q_b, \text{chk}_a, \text{chk}_b, \text{back}, t, r\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{a, b, X, \sqcup\}$
- δ given by:

δ	a	b	X	$\sqcup$
s	$(q_a, X, +1)$	$(q_b, X, +1)$	$(s, X, +1)$	$(t, \sqcup, +1)$
q_a	$(q_a, a, +1)$	$(q_a, b, +1)$	$(q_a, X, +1)$	$(\text{chk}_a, \sqcup, -1)$
q_b	$(q_b, a, +1)$	$(q_b, b, +1)$	$(q_b, X, +1)$	$(\text{chk}_b, \sqcup, -1)$
chk_a	$(\text{back}, X, -1)$	$(r, b, +1)$	$(\text{chk}_a, X, -1)$	$(t, \sqcup, +1)$
chk_b	$(r, a, +1)$	$(\text{back}, X, -1)$	$(\text{chk}_b, X, -1)$	$(t, \sqcup, +1)$
back	$(\text{back}, a, -1)$	$(\text{back}, b, -1)$	$(s, X, +1)$	$(t, \sqcup, +1)$

Reading the table row by row:

- s: Read $a \to$ remember it (go to q_a), mark cell with X. Read $b \to$ similar. Read $X \to$ skip already-used cells. Read $\sqcup \to$ all letters consumed, accept.
- q_a/q_b: Scan right past everything. When hitting $\sqcup$, go left one step into the check state.
- chk_a: We're looking at the last unused letter and need it to be a. If a: match! Mark with X, go to back. If b: mismatch, reject. If X: skip (already used), keep going left. If $\sqcup$: the word was length 1 (just the letter we already consumed), accept.
- chk_b: Same but needs b.
- back: Scan left to find the start of the remaining input (X marks the boundary), then go right one step into s to start the next round.

2. Run on aba (palindrome, should accept):

1. $[s, aba \sqcup \ldots, 1]$: $\delta(s, a) = (q_a, X, +1)$
2. $[q_a, Xba \sqcup \ldots, 2]$: $\delta(q_a, b) = (q_a, b, +1)$
3. $[q_a, Xba \sqcup \ldots, 3]$: $\delta(q_a, a) = (q_a, a, +1)$
4. $[q_a, Xba \sqcup \ldots, 4]$: $\delta(q_a, \sqcup) = (\text{chk}_a, \sqcup, -1)$
5. $[\text{chk}_a, Xba \sqcup \ldots, 3]$: $\delta(\text{chk}_a, a) = (\text{back}, X, -1)$. Match!
6. $[\text{back}, XbX \sqcup \ldots, 2]$: $\delta(\text{back}, b) = (\text{back}, b, -1)$
7. $[\text{back}, XbX \sqcup \ldots, 1]$: $\delta(\text{back}, X) = (s, X, +1)$. New round.
8. $[s, XbX \sqcup \ldots, 2]$: $\delta(s, b) = (q_b, X, +1)$
9. $[q_b, XXX \sqcup \ldots, 3]$: $\delta(q_b, X) = (q_b, X, +1)$
10. $[q_b, XXX \sqcup \ldots, 4]$: $\delta(q_b, \sqcup) = (\text{chk}_b, \sqcup, -1)$
11. $[\text{chk}_b, XXX \sqcup \ldots, 3]$: $\delta(\text{chk}_b, X) = (\text{chk}_b, X, -1)$. Skip X.
12. $[\text{chk}_b, XXX \sqcup \ldots, 2]$: $\delta(\text{chk}_b, X) = (\text{chk}_b, X, -1)$. Skip X.
13. $[\text{chk}_b, XXX \sqcup \ldots, 1]$: $\delta(\text{chk}_b, X) = (\text{chk}_b, X, -1)$. Skip X.
14. $[\text{chk}_b, XXX \sqcup \ldots, 1]$: Head at position 1, direction -1, so head **stays** at 1. Wait — we need to check: position is already 1 and we try to go left. Head stays at 1.

    Actually, let me redo this more carefully. After step 10, we're at position 3 reading X. Step 11 goes to position 2 reading X. Step 12 goes to position 1 reading X. Step 13 tries to go to position 0 but head stays at 1.

    Hmm, this reveals a subtlety. Let me reconsider the chk_b row for X: it goes left. But what if we reach the left boundary? The $\sqcup$ case handles that: $\delta(\text{chk}_b, \sqcup) = (t, \sqcup, +1)$. But we never reach $\sqcup$ because position 1 has X, not $\sqcup$.

    Fix: The issue is that chk_b skips X's going left, but it might get stuck at the left boundary on an X. We need to handle this: if chk_b keeps seeing only X's and reaches the boundary, the word had only one letter left (already consumed), so we should accept.

    The head-stays-at-position-1 special case means we keep reading X at position 1 forever. This is a bug in the construction!

    Better fix: Instead of skipping X's in chk, we should have the q_a/q_b states stop at the last *non-X* letter rather than at $\sqcup$. Or we handle the boundary differently.

    Simplest fix: Change back to go to s when it sees X at position 1. But actually, a cleaner design: when scanning right in q_a/q_b, stop before $\sqcup$ but also skip X's. Then chk doesn't need to skip X's. Let me redo the transitions for q_a:

    Actually, the simplest robust approach: in chk_a and chk_b, add the rule that if we're skipping X's and see another X after going left from an X at the boundary (i.e., head doesn't move), we accept. But DTMs don't detect "head didn't move" directly.

    The proper fix: change $\delta(\text{chk}_a, X) = (t, X, +1)$ and $\delta(\text{chk}_b, X) = (t, X, +1)$ — if after going left past $\sqcup$ we find X, it means all remaining cells are used, so accept.

    Wait no, the issue is different. Let me reconsider.

After step 8, the tape is $XXX\sqcup\ldots$. We're scanning right from position 2 in q_b. Position 3 is X, position 4 is $\sqcup$. So chk_b lands on position 3, sees X. Should go left. Position 2 is X. Position 1 is X. Position 1, try to go left, stay at 1, still see X.

The problem: all remaining letters are X (already consumed). The middle letter b was consumed. Since it was the only letter left and we already "remembered" it, we should accept.

Correct approach: The check states should accept when they only see X's (nothing left to compare against $\Rightarrow$ the single middle letter is fine).

Revised: $\delta(\text{chk}_a, X) = (t, X, -1)$ and $\delta(\text{chk}_b, X) = (t, X, -1)$.

With this fix, step 11: $[\text{chk}_b, XXX\sqcup\ldots, 3]$, read X, accept.

Let me give the clean corrected δ table:

δ	a	b	X	$\sqcup$
s	$(q_a, X, +1)$	$(q_b, X, +1)$	$(s, X, +1)$	$(t, \sqcup, +1)$
q_a	$(q_a, a, +1)$	$(q_a, b, +1)$	$(q_a, X, +1)$	$(\text{chk}_a, \sqcup, -1)$
q_b	$(q_b, a, +1)$	$(q_b, b, +1)$	$(q_b, X, +1)$	$(\text{chk}_b, \sqcup, -1)$
chk_a	$(\text{back}, X, -1)$	$(r, b, +1)$	$(t, X, +1)$	$(t, \sqcup, +1)$
chk_b	$(r, a, +1)$	$(\text{back}, X, -1)$	$(t, X, +1)$	$(t, \sqcup, +1)$
back	$(\text{back}, a, -1)$	$(\text{back}, b, -1)$	$(s, X, +1)$	$(r, \sqcup, +1)$

The change: chk_a and chk_b reading X now accept (instead of skipping left), because if the last non-blank cell is an X, all letters have been consumed and matched.

Corrected run on aba:
$[s, aba\sqcup\ldots, 1] \vdash_M [q_a, Xba\sqcup\ldots, 2] \vdash_M [q_a, Xba\sqcup\ldots, 3] \vdash_M [q_a, Xba\sqcup\ldots, 4]$
$\vdash_M [\text{chk}_a, Xba\sqcup\ldots, 3] \vdash_M [\text{back}, XbX\sqcup\ldots, 2] \vdash_M [\text{back}, XbX\sqcup\ldots, 1]$
$\vdash_M [s, XbX\sqcup\ldots, 2] \vdash_M [q_b, XXX\sqcup\ldots, 3] \vdash_M [q_b, XXX\sqcup\ldots, 4]$
$\vdash_M [\text{chk}_b, XXX\sqcup\ldots, 3] \vdash_M [t, XXX\sqcup\ldots, 4]$
aba is **accepted**. ✓

3. Run on ab (not a palindrome, should reject):
$[s, ab\sqcup\ldots, 1] \vdash_M [q_a, Xb\sqcup\ldots, 2] \vdash_M [q_a, Xb\sqcup\ldots, 3] \vdash_M [\text{chk}_a, Xb\sqcup\ldots, 2]$
$\vdash_M [r, Xb\sqcup\ldots, 3]$
chk_a reads b, but needs a. Mismatch $\rightarrow$ reject. ✓

What to learn from this exercise

- **Start with the algorithm in words.** The palindrome algorithm is: compare first and last, shrink, repeat.
- **States encode memory.** q_a = "first letter was a". q_b = "first letter was b".
- **Tape marks track progress.** Writing X = "this letter has been matched."
- **Corner cases WILL bite you.** The original construction had a bug with all-X tapes. Always trace your DTM on corner cases (ε, single letter, short inputs).
- **Showing a bug and fixing it is totally fine.** The solution above found a problem during tracing and fixed it. This is the normal process — even in exams, if you catch and correct an error, it shows understanding.

Exercise 6: What Language Does This DTM Accept?

Consider the DTM $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ with:

- $Q = \{s, t, r, q\}$
- $\Sigma = \{a\}$
- $\Gamma = \{a, \sqcup\}$
- δ given by:

δ	a	$\sqcup$
s	$(q, \sqcup, +1)$	$(r, \sqcup, +1)$
q	$(s, \sqcup, +1)$	$(t, \sqcup, +1)$

1. Give the run on ε, a, aa, aaa, and $aaaa$.
2. Based on the runs, determine what language M accepts.
3. Is M a halting DTM?

Solution to Exercise 6

1. Runs:
Run on ε: $[s, \sqcup \ldots, 1] \vdash_M [r, \sqcup \ldots, 2]$. **Rejected.**
Run on a: $[s, a\sqcup \ldots, 1] \vdash_M [q, \sqcup\sqcup \ldots, 2] \vdash_M [t, \sqcup \ldots, 3]$. **Accepted.**
Run on aa: $[s, aa\sqcup \ldots, 1] \vdash_M [q, \sqcup a\sqcup \ldots, 2] \vdash_M [s, \sqcup\sqcup\sqcup \ldots, 3] \vdash_M [r, \sqcup \ldots, 4]$. **Rejected.**
Run on aaa: $[s, aaa\sqcup \ldots, 1] \vdash_M [q, \sqcup aa\sqcup \ldots, 2] \vdash_M [s, \sqcup\sqcup a\sqcup \ldots, 3] \vdash_M [q, \sqcup\sqcup\sqcup\sqcup \ldots, 4] \vdash_M [t, \sqcup \ldots, 5]$. **Accepted.**
Run on $aaaa$: $[s, aaaa\sqcup \ldots, 1] \vdash_M [q, \sqcup aaa\sqcup \ldots, 2] \vdash_M [s, \sqcup\sqcup aa\sqcup \ldots, 3] \vdash_M [q, \sqcup\sqcup\sqcup a\sqcup \ldots, 4] \vdash_M [s, \sqcup\sqcup\sqcup\sqcup\sqcup \ldots, 5] \vdash_M [r, \sqcup \ldots, 6]$
Rejected.

2. Pattern:

Input	Result
ε (length 0)	Reject
a (length 1)	Accept
aa (length 2)	Reject
aaa (length 3)	Accept
$aaaa$ (length 4)	Reject

M accepts words of **odd** length: $L(M) = \{a^n \mid n \text{ is odd}, n \geq 1\}$.
The machine alternates between s and q while consuming letters. It accepts if it finishes in q (odd number of letters read), rejects if it finishes in s (even number, including 0).

3. Yes, M is a **halting** DTM. The head moves right on every step and eventually reaches $\sqcup$ in either s (reject) or q (accept). The machine always terminates.
Since M is halting, $L(M)$ is a **computable** language.

After completing this sheet:

Go back to the *original* Tutorial 1 exercises (without solutions).
Try Exercise 1 (Test your understanding) and Exercise 3 (Traces) first.
They should feel familiar now.

Then try Exercise 4 (Constructing a simple DTM) — use the same 5-phase method you saw in Exercise 4 and 5 above.